# Informe técnico · AsesorIA Planes

*Pruebas E2E automatizadas con un agente de IA y TestSprite sobre un MVP con pasarela de pagos*

| Estudiante | Fecha | Caso de estudio | Herramientas |
|---|---|---|---|
| Manuela Larrea | 20 de octubre de 2026 | AsesorIA Planes · portal de suscripción con pasarela de pagos (MVP propio, continuación del proyecto del curso) | Cursor + agente de IA · Next.js 15 · Stripe Checkout (test) · Vercel · GitHub Actions · TestSprite (MCP) · Playwright 1.56 |

## 1. Caso de estudio y enfoque

El caso analizado es propio: AsesorIA, el asistente de IA para asesores financieros desarrollado durante el curso, necesita un canal de venta por suscripción. El flujo de pago es el escenario E2E más exigente de una aplicación web porque atraviesa tres dominios de confianza (la aplicación, la página alojada de la pasarela y el servidor que verifica el pago) y porque los fallos tienen costo económico y regulatorio. Por eso se eligió para evaluar la automatización de pruebas E2E con agentes de IA: un agente debe ser capaz de leer la aplicación, derivar un plan de pruebas a partir de los requisitos, ejecutarlo contra un entorno real y explicar los fallos sin intervención manual.

La estrategia se diseñó en dos capas. La capa determinista corre en cualquier entorno sin cuentas externas: la app expone un proveedor de pagos simulado (mock) con sesiones firmadas que reproduce la página alojada de Stripe, de modo que el plan de pruebas completo se ejecuta en CI en 14 segundos. La capa real usa Stripe en modo test sobre el despliegue de Vercel, donde TestSprite ejecuta el mismo plan desde la nube. Esta separación es una buena práctica de testabilidad: las pruebas de IA se vuelven reproducibles y no dependen de la disponibilidad ni del coste de la pasarela.

## 2. La aplicación (MVP): flujo, pasarela y despliegue

| Capa | Implementación | Decisión de diseño |
|---|---|---|
| Frontend / páginas | Next.js 15 App Router (TypeScript, React 19): /, /planes, /checkout, /checkout/mock, /exito, /cancelado, /cuenta | Server Components para datos sensibles; estado del cliente mínimo; data-testid y nombres accesibles en todo elemento interactivo |
| Pasarela de pagos | lib/payments.ts: interfaz PaymentProvider con StripeProvider (Checkout en modo suscripción, success/cancel URL, metadata del plan, webhook firmado) y MockProvider (sesión HMAC stateless, página simulada con tarjeta 4242…) | La app nunca ve datos de tarjeta; la confirmación se verifica en el servidor con retrieve antes de mostrar éxito o activar |
| Activación y cuenta | POST /api/activar valida paymentStatus=paid y emite una cookie firmada (HMAC) con plan, correo y referencia; /cuenta la lee; DELETE la revoca | Sin base de datos en el MVP: el estado viaja firmado; un intento de activar sin pago responde 402 |
| API y observabilidad | /api/health (proveedor, commit), /api/session/{id}, /api/webhook (Stripe), /api/mock/complete | Contrato estable para pruebas de API y smoke tras el despliegue |
| Generación con IA | Código generado por un agente de IA y refinado en Cursor con reglas de proyecto (.cursor/rules) que fijan convenciones de testabilidad y seguridad | Las reglas evitan que una regeneración rompa los testids del plan de pruebas |
| Despliegue automatizado | Vercel (Root Directory asesoria-planes) + workflow deploy-planes.yml: E2E con mock → vercel deploy --prod → smoke contra la URL pública | El despliegue solo ocurre si el plan de pruebas pasa; el smoke valida salud, catálogo y API en producción |

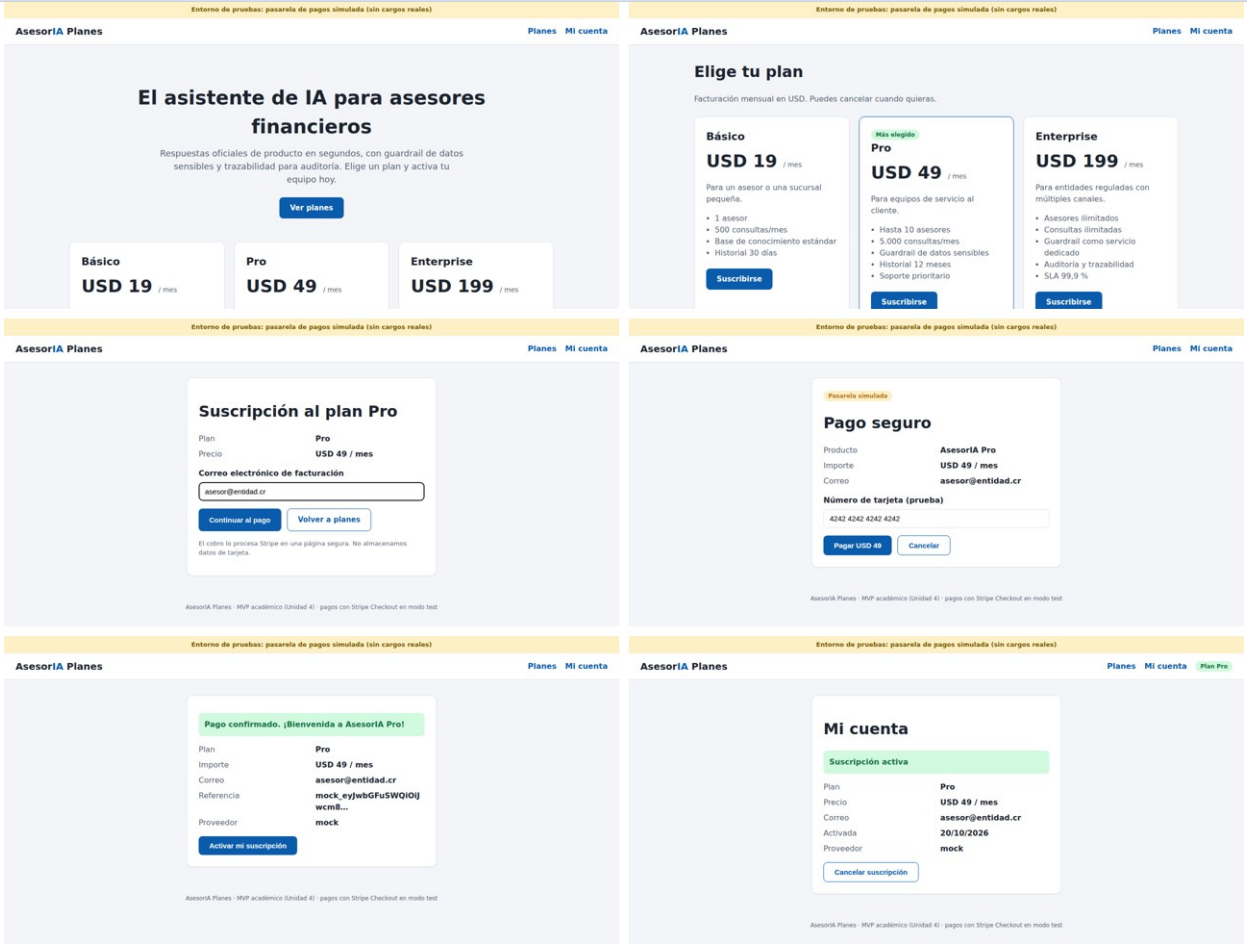



*Figura 1. Flujo del MVP en modo de pruebas: landing, catálogo, checkout, pasarela simulada, confirmación verificada y cuenta con plan activo.*

Smoke de la API del MVP (proveedor mock) tras el build de producción

```
$ curl -s localhost:3100/api/health
{"status":"ok","app":"asesoria-planes","version":"0.1.0","paymentsProvider":"mock","planes":["basico","pro","enterprise"],"commit":"loc
$ curl -s -H 'Content-Type: application/json' -d '{"planId":"pro","email":"asesor@entidad.cr"}' localhost:3100/api/checkout
{"sessionId":"mock_eyJwbGFuSWQiOiJwcm8iLCJlbWFpbCI6ImFzZXNvckBlbnRpZGFkLmNyIiwic3RhdHVzIjoib3BlbiJ9.d9d0df27feb4f76f","url":"http://loc
$ curl -s -H 'Content-Type: application/json' -d '{"sessionId":"mock_…","outcome":"pay"}' localhost:3100/api/mock/complete
{"sessionId":"mock_eyJwbGFuSWQiOiJwcm8iLCJlbWFpbCI6ImFzZXNvckBlbnRpZGFkLmNyIiwic3RhdHVzIjoiY29tcGxldGUifQ.9a707843f225ca06","status":"c
$ curl -s localhost:3100/api/session/mock_…complete…
{"id":"mock_…","provider":"mock","status":"complete","paymentStatus":"paid","planId":"pro","email":"asesor@entidad.cr","amountUsd":49}
$ curl -s -c cookies -H 'Content-Type: application/json' -d '{"sessionId":"mock_…complete…"}' localhost:3100/api/activar
{"activada":true,"planId":"pro"}
$ curl -s -H 'Content-Type: application/json' -d '{"planId":"pro","email":"malo"}' -w ' %{http_code}' localhost:3100/api/checkout
{"error":"Correo electrónico inválido."} 400
$ npx next build   →   14 rutas compiladas (ƒ dynamic) · tsc OK · eslint sin hallazgos
```

*Figura 2. Build de producción y smoke del contrato de la API: creación de sesión, confirmación, verificación, activación y validación de correo.*

```ts
/**
 * Capa de pagos con dos proveedores intercambiables:
 *   - StripeProvider: Stripe Checkout (modo test) — producción/staging.
 *   - MockProvider:   simula la página alojada de pago sin red; se usa en pruebas E2E y en CI.
 * Selección: PAYMENTS_PROVIDER=stripe|mock (por defecto, stripe si existe STRIPE_SECRET_KEY).
 */
import { createHmac, timingSafeEqual } from "node:crypto";
import Stripe from "stripe";
import { getPlan, type Plan } from "./plans";

export type CheckoutStatus = "complete" | "open" | "expired";

export interface CheckoutSession {
  id: string;
  url: string;                 // URL de la página de pago alojada
  provider: "stripe" | "mock";
}

export interface SessionDetails {
  id: string;
  provider: "stripe" | "mock";
  status: CheckoutStatus;
  paymentStatus: "paid" | "unpaid" | "no_payment_required";
  planId: string;
  email: string;
  amountUsd: number;
}

export interface PaymentProvider {
  readonly name: "stripe" | "mock";
  createCheckout(plan: Plan, email: string, baseUrl: string): Promise<CheckoutSession>;
  retrieve(sessionId: string): Promise<SessionDetails | null>;
}

function normalizePaymentStatus(value: string): SessionDetails["paymentStatus"] {
  return value === "paid" || value === "no_payment_required" ? value : "unpaid";
}

// ---------------------------------------------------------------- Stripe
class StripeProvider implements PaymentProvider {
  readonly name = "stripe" as const;
  private readonly stripe: Stripe;

  constructor(secretKey: string) {
    this.stripe = new Stripe(secretKey);
  }

  async createCheckout(plan: Plan, email: string, baseUrl: string): Promise<CheckoutSession> {
    const priceId = process.env[plan.stripePriceEnv];
    const session = await this.stripe.checkout.sessions.create({
      mode: "subscription",
      customer_email: email,
      line_items: [
        priceId
          ? { price: priceId, quantity: 1 }
          : {
              quantity: 1,
              price_data: {
                currency: "usd",
                unit_amount: plan.precioUsd * 100,
```

```ts
export interface Strategies {
  testId?: string;
  role?: { role: Parameters<Page["getByRole"]>[0]; name?: string | RegExp };
  text?: string | RegExp;
  label?: string | RegExp;
  css?: string;
}

export interface HealingEvent {
  test: string; element: string; failed: string; healed: string; at: string;
}

const LOG = "test-results/healing-log.json";

function record(event: HealingEvent) {
  mkdirSync("test-results", { recursive: true });
  appendFileSync(LOG, JSON.stringify(event) + "\n");
}

function candidates(page: Page, s: Strategies): Array<[string, Locator]> {
  const list: Array<[string, Locator]> = [];
  if (s.testId) list.push([`testId=${s.testId}`, page.getByTestId(s.testId)]);
  if (s.role) list.push([`role=${s.role.role}${s.role.name ? ` name=${s.role.name}` : ""}`, page.getByRole(s.role.role, { name: s.role.name })]);
  if (s.label) list.push([`label=${s.label}`, page.getByLabel(s.label)]);
  if (s.text) list.push([`text=${s.text}`, page.getByText(s.text)]);
  if (s.css) list.push([`css=${s.css}`, page.locator(s.css)]);
  return list;
}

/**
 * Devuelve el primer localizador que resuelve a exactamente un elemento visible dentro del
 * tiempo indicado. Si el primario falla pero otro funciona, lo registra como curación.
 */
export async function heal(page: Page, element: string, s: Strategies, opts: { test?: string; timeout?: number } = {}): Promise<Locator> {
  const timeout = opts.timeout ?? 3000;
  const list = candidates(page, s);
  let primary: string | undefined;
  for (const [name, locator] of list) {
    primary ??= name;
    try {
      await expect(locator.first()).toBeVisible({ timeout });
      if (name !== primary) record({ test: opts.test ?? "", element, failed: primary, healed: name, at: new Date().toISOString() });
      return locator.first();
    } catch {
      // probar la siguiente estrategia
    }
  }
  throw new Error(`Elemento "${element}" no encontrado con ninguna estrategia: ${list.map(([n]) => n).join(" | ")}`);
}
```

*Figura 3. Código generado y refinado con IA: lib/payments.ts (interfaz PaymentProvider con Stripe y mock) y tests/support/healing.ts (localizadores auto-curables).*

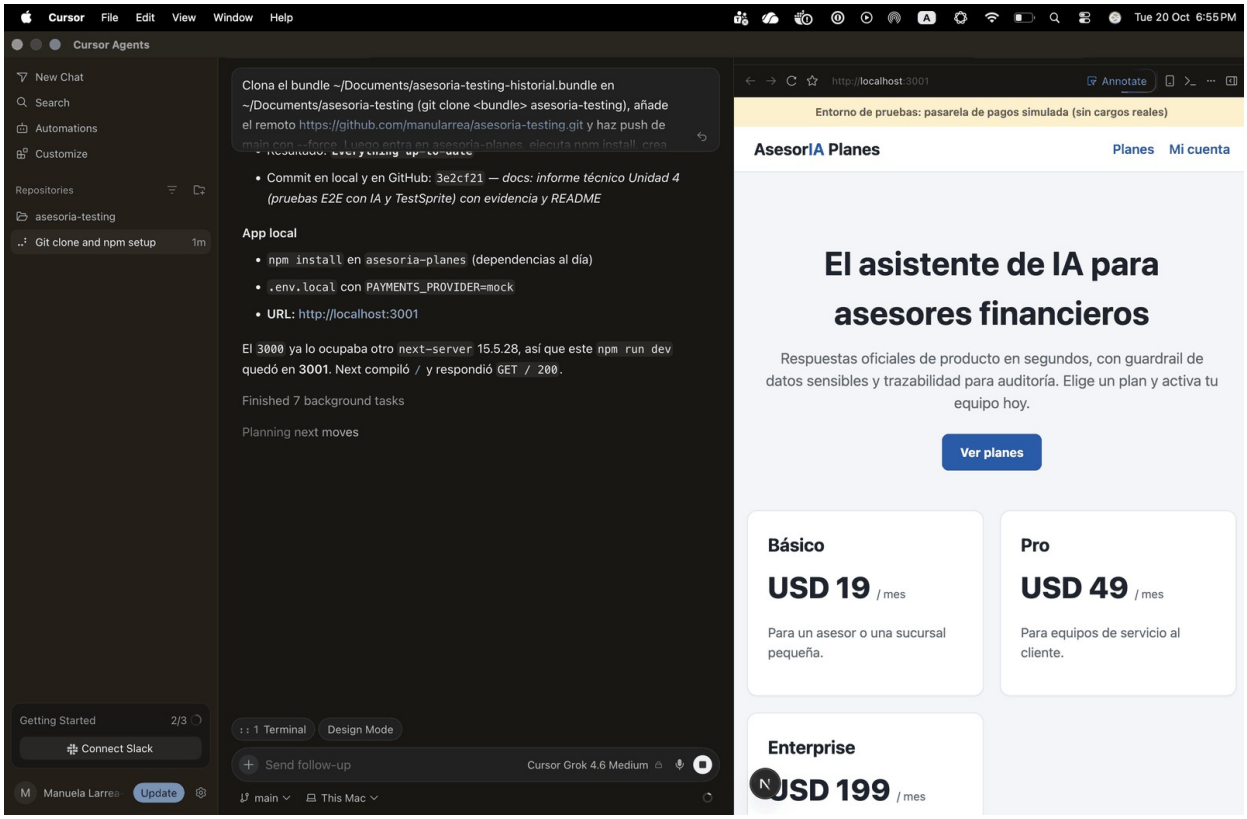



*Figura 4. Agente de Cursor (Grok 4.6) sobre el repositorio asesoria-testing: clona, instala dependencias, crea .env.local con el proveedor mock y levanta la app en localhost:3001 (navegador integrado a la derecha).*

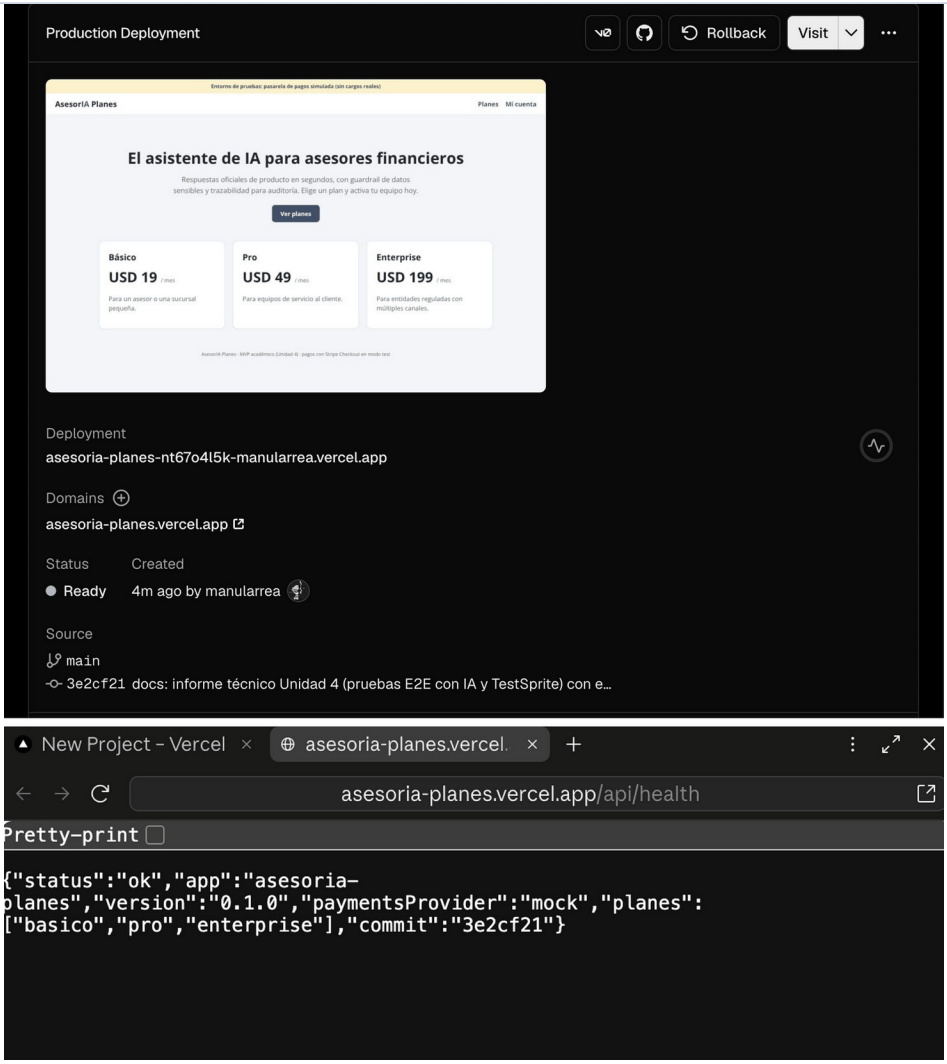



*Figura 5. Despliegue en Vercel: proyecto asesoria-planes (Root Directory asesoria-planes, rama main, commit 3e2cf21) en estado Ready en https://asesoria-planes.vercel.app y respuesta de /api/health en producción.*

## 3. Plan de pruebas generado con IA e integración con TestSprite

TestSprite es una plataforma de pruebas con agentes de IA que se integra en el IDE mediante un servidor MCP (@testsprite/testsprite-mcp): a partir del código y de un PRD genera un plan de pruebas, escribe y ejecuta los casos en su nube contra la aplicación (local por túnel o URL pública), y devuelve un reporte con causa raíz y sugerencias de corrección que el propio agente del IDE puede aplicar. La integración se preparó en tres artefactos del repositorio: la configuración del MCP (.cursor/mcp.json), el PRD estándar (testsprite_tests/standard_prd.json: visión, seis funcionalidades, flujos de usuario y criterios de validación) y el plan de pruebas en el formato de TestSprite (testsprite_frontend_test_plan.json). Cada caso se implementó además en Playwright (tests/e2e) para poder ejecutarlo en CI y comparar los resultados de ambos ejecutores sobre el mismo plan.

| ID | Caso | Categoría | Prioridad | Qué valida |
|---|---|---|---|---|
| TC001 | La landing presenta la propuesta de valor, el CTA y el resumen de los tres planes | functional | High | Verifica que la página de inicio carga con el título, el botón Ver planes y tres tarjetas con los precios del catálogo. |
| TC002 | El catálogo lista los planes con precios correctos y destaca el plan Pro | functional | High | Comprueba contenido y precios del catálogo y que cada plan tiene su botón Suscribirse. |
| TC003 | El checkout muestra el plan y el precio seleccionados | functional | High | Desde el catálogo, Suscribirse en Pro lleva al checkout con el resumen correcto. |
| TC004 | El checkout rechaza un correo inválido sin redirigir a la pasarela | error handling | High | Validación de entrada: correo malformado produce error accesible y no se crea sesión. |
| TC005 | Un plan inexistente en el checkout devuelve 404 | error handling | Medium | Robustez ante parámetros manipulados en la URL. |

| ID | Caso | Categoría | Prioridad | Qué valida |
|---|---|---|---|---|
| TC006 | Flujo de pago exitoso de extremo a extremo (checkout → pasarela → confirmación) | functional | High | Caso principal: con un correo válido y la tarjeta de prueba el usuario llega a /exito con plan, importe, correo y referencia. |
| TC007 | Activar la suscripción tras el pago muestra el plan en Mi cuenta y en la cabecera | functional | High | La activación persiste la suscripción (cookie firmada) y la UI la refleja. |
| TC008 | Cancelar en la pasarela lleva a /cancelado y permite reintentar el mismo plan | functional | Medium | Ruta de cancelación: no hay cargo ni activación. |
| TC009 | La pasarela rechaza una tarjeta inválida y no confirma el pago | error handling | High | Tarjeta distinta de la de prueba produce error y permanece en la pasarela. |
| TC010 | Una sesión de pago inválida o manipulada no muestra la confirmación ni activa la suscripción | security | High | Protección contra acceso directo a /exito y contra activación sin pago. |
| TC011 | Contrato de la API: health, checkout y sesión | api | Medium | Validaciones y respuestas de los endpoints que usa el frontend. |
| TC012 | Cancelar la suscripción desde Mi cuenta elimina el plan activo | functional | Medium | Ciclo completo de la suscripción. |

## Flujo de integración con TestSprite desde Cursor:

| Paso | Acción en Cursor + TestSprite | Resultado |
|---|---|---|
| 1 | Configurar el MCP con la API key y levantar la app (PAYMENTS_PROVIDER=mock, puerto 3000) | Servidor MCP disponible en el chat de Cursor |
| 2 | Prompt: «Help me test this project with TestSprite» → frontend, puerto 3000, alcance codebase | Bootstrap: TestSprite lee el código y pide/genera el PRD |
| 3 | Revisar standard_prd.json y testsprite_frontend_test_plan.json (comparar con los del repositorio) | Plan de 12 casos aprobado |
| 4 | Ejecutar el plan desde el chat | Casos ejecutados en la nube con video; testsprite-mcp-test-report.md |
| 5 | Pedir al agente que corrija los fallos reportados y re-ejecutar | Ciclo de auto-corrección (self-healing asistido por IA) |
| 6 | Repetir contra la URL de Vercel con Stripe en modo test | Validación del flujo real hasta la página de Stripe Checkout |



```
{
  "mcpServers": {
    "TestSprite": {
      "command": "npx",
      "args": ["@testsprite/testsprite-mcp@latest"],
      "env": { "API_KEY": "PEGAR_AQUI_TU_API_KEY_DE_TESTSPRITE" }
    }
  }
}
```

```
---
description: Convenciones del MVP AsesorIA Planes (Next.js 15 + Stripe) para generación y pruebas con IA
globs: ["**/*.ts", "**/*.tsx"]
alwaysApply: true
---
- Stack: Next.js 15 App Router, TypeScript estricto, React 19, sin librerías de UI; estilos en app/globals.css.
- Pagos: toda interacción con la pasarela pasa por lib/payments.ts (PaymentProvider). Nunca llamar a Stripe desde componentes.
  El proveedor mock (PAYMENTS_PROVIDER=mock) debe seguir funcionando: es el que usan TestSprite/Playwright en CI.
- Seguridad: no almacenar datos de tarjeta; verificar en el servidor (retrieve) antes de mostrar éxito o activar; cookies firmadas.
- Testabilidad: todo elemento interactivo o de resultado lleva data-testid estable en español (p. ej. checkout-pagar, exito-plan)
  y un nombre accesible; los errores usan role="alert". No cambiar testids existentes sin actualizar tests/e2e y el plan de TestSprite.
- Pruebas: tests/e2e implementa testsprite_tests/testsprite_frontend_test_plan.json (TC001..TC012). Un cambio funcional actualiza
  primero el plan (id, título, pasos) y luego la prueba. Usar tests/support/healing.ts para localizadores con alternativas.
- Idioma de UI y mensajes: español neutro (Costa Rica). Moneda: "USD 49".
```

```
[
  {"id": "TC001", "title": "La landing presenta la propuesta de valor, el CTA y el resumen de los tres planes", "category": "functional", "priority":
   "description": "Verifica que la página de inicio carga con el título, el botón Ver planes y tres tarjetas con los precios del catálogo.",
   "steps": [{"type": "action", "description": "Abrir /"}, {"type": "assertion", "description": "Se muestra el título principal y el botón Ver planes"
  {"id": "TC002", "title": "El catálogo lista los planes con precios correctos y destaca el plan Pro", "category": "functional", "priority": "High",
   "description": "Comprueba contenido y precios del catálogo y que cada plan tiene su botón Suscribirse.",
   "steps": [{"type": "action", "description": "Abrir /planes"}, {"type": "assertion", "description": "Básico USD 19, Pro USD 49, Enterprise USD 199"},
  {"id": "TC003", "title": "El checkout muestra el plan y el precio seleccionados", "category": "functional", "priority": "High",
   "description": "Desde el catálogo, Suscribirse en Pro lleva al checkout con el resumen correcto.",
   "steps": [{"type": "action", "description": "Abrir /planes y pulsar Suscribirse del plan Pro"}, {"type": "assertion", "description": "URL contiene
  {"id": "TC004", "title": "El checkout rechaza un correo inválido sin redirigir a la pasarela", "category": "error handling", "priority": "High",
   "description": "Validación de entrada: correo malformado produce error accesible y no se crea sesión.",
   "steps": [{"type": "action", "description": "Abrir /checkout?plan=pro"}, {"type": "action", "description": "Escribir 'correo-invalido' y pulsar Con
  {"id": "TC005", "title": "Un plan inexistente en el checkout devuelve 404", "category": "error handling", "priority": "Medium",
```

*Figura 6. Integración con TestSprite: .cursor/mcp.json (servidor MCP), reglas del proyecto .cursor/rules y plan de pruebas testsprite_frontend_test_plan.json generado a partir del PRD.*

**[PENDIENTE] Captura de TestSprite**

Dashboard de testsprite.com con el resumen de la ejecución (pasados/fallidos) y el detalle de un caso con video; testsprite_tests/testsprite-mcp-test-report.md generado desde Cursor. Guía §4.

*Figura 7. Ejecución del plan en TestSprite (captura pendiente; ver Anexo).*

## 4. Revisión y validación de cada caso

Cada caso generado se revisó contra el PRD antes de ejecutarlo y, tras la primera ejecución, se clasificó la causa de cada fallo como defecto del producto, expectativa incorrecta de la prueba o fragilidad del localizador. La tabla resume el ciclo; los logs completos están en testsprite_tests/ejecucion_1_fallos.log y ejecucion_2_final.log.

| ID | Resultado 1.ª ejecución | Causa raíz | Corrección | Final |
|---|---|---|---|---|
| TC001 | ✘ | Aserción toContainText con lista sobre un solo elemento; formato de moneda $19 vs USD 19 | Aserción por tarjeta; formato explícito USD | ✓ |
| TC002 | ✘ | Formato de moneda (Intl es-CR produce «USD 19», la prueba esperaba «$19») | formatUsd determinista «USD 19» | ✓ |
| TC003 | ✓ (curado) | testid primario obsoleto (btn-subscribe-pro, simulado) | heal() resolvió por rol+nombre y lo registró | ✓ |
| TC004 | ✘ | getByRole('alert') ambiguo: Next.js inyecta <next-route-announcer role=alert> | Localizador por data-testid; patrón documentado | ✓ |
| TC005 | ✓ | — | — | ✓ |
| TC006 | ✘ | Formato de moneda en la pasarela; testid btn-pay obsoleto (curado) | Formato USD; curación por rol «Pagar» | ✓ |
| TC007 | ✘ | Defecto de producto: tras activar, router.push no refresca el layout y la cabecera no muestra el plan | router.refresh() en ActivarButton | ✓ |
| TC008–TC012 | ✓ | — | — | ✓ |

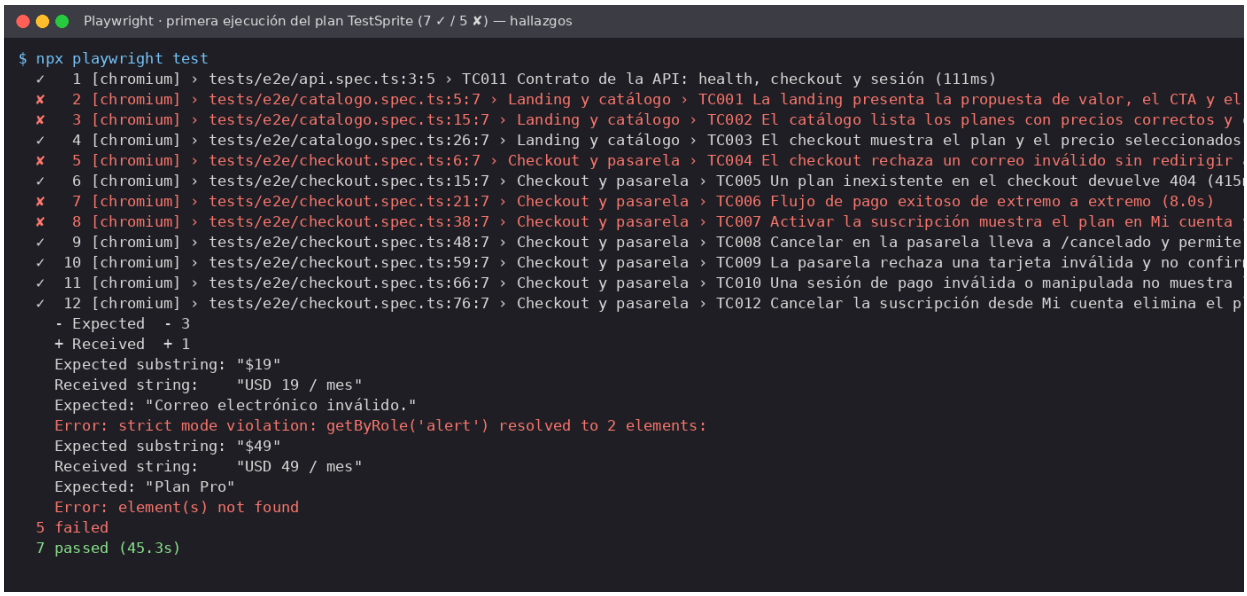



*Figura 8. Primera ejecución del plan: 7 casos pasan, 5 fallan; se muestran las aserciones y errores de localización que originaron la revisión.*

Playwright · ejecución final tras correcciones (12/12) + registro de auto-curación

```
$ npx playwright test
  ✓   1 [chromium] › tests/e2e/api.spec.ts:3:5 › TC011 Contrato de la API: health, checkout y sesión (93ms)
  ✓   2 [chromium] › tests/e2e/catalogo.spec.ts:5:7 › Landing y catálogo › TC001 La landing presenta la propuesta de valor, el CTA y el
  ✓   3 [chromium] › tests/e2e/catalogo.spec.ts:15:7 › Landing y catálogo › TC002 El catálogo lista los planes con precios correctos y
  ✓   4 [chromium] › tests/e2e/catalogo.spec.ts:26:7 › Landing y catálogo › TC003 El checkout muestra el plan y el precio seleccionados
  ✓   5 [chromium] › tests/e2e/checkout.spec.ts:6:7 › Checkout y pasarela › TC004 El checkout rechaza un correo inválido sin redirigir
  ✓   6 [chromium] › tests/e2e/checkout.spec.ts:15:7 › Checkout y pasarela › TC005 Un plan inexistente en el checkout devuelve 404 (317
  ✓   7 [chromium] › tests/e2e/checkout.spec.ts:21:7 › Checkout y pasarela › TC006 Flujo de pago exitoso de extremo a extremo (3.9s)
  ✓   8 [chromium] › tests/e2e/checkout.spec.ts:38:7 › Checkout y pasarela › TC007 Activar la suscripción muestra el plan en Mi cuenta
  ✓   9 [chromium] › tests/e2e/checkout.spec.ts:48:7 › Checkout y pasarela › TC008 Cancelar en la pasarela lleva a /cancelado y permite
  ✓  10 [chromium] › tests/e2e/checkout.spec.ts:59:7 › Checkout y pasarela › TC009 La pasarela rechaza una tarjeta inválida y no confir
  ✓  11 [chromium] › tests/e2e/checkout.spec.ts:66:7 › Checkout y pasarela › TC010 Una sesión de pago inválida o manipulada no muestra
  ✓  12 [chromium] › tests/e2e/checkout.spec.ts:76:7 › Checkout y pasarela › TC012 Cancelar la suscripción desde Mi cuenta elimina el p
  12 passed (14.0s)

$ cat test-results/healing-log.json
{"test":"TC003 El checkout muestra el plan y el precio seleccionados","element":"botón Suscribirse (Pro)","failed":"testId=btn-subscrib
{"test":"TC006 Flujo de pago exitoso de extremo a extremo","element":"botón Pagar","failed":"testId=btn-pay","healed":"role=button name
```

*Figura 9. Ejecución final: 12/12 en 14 s y registro de las dos curaciones de localizadores (TC003 y TC006).*

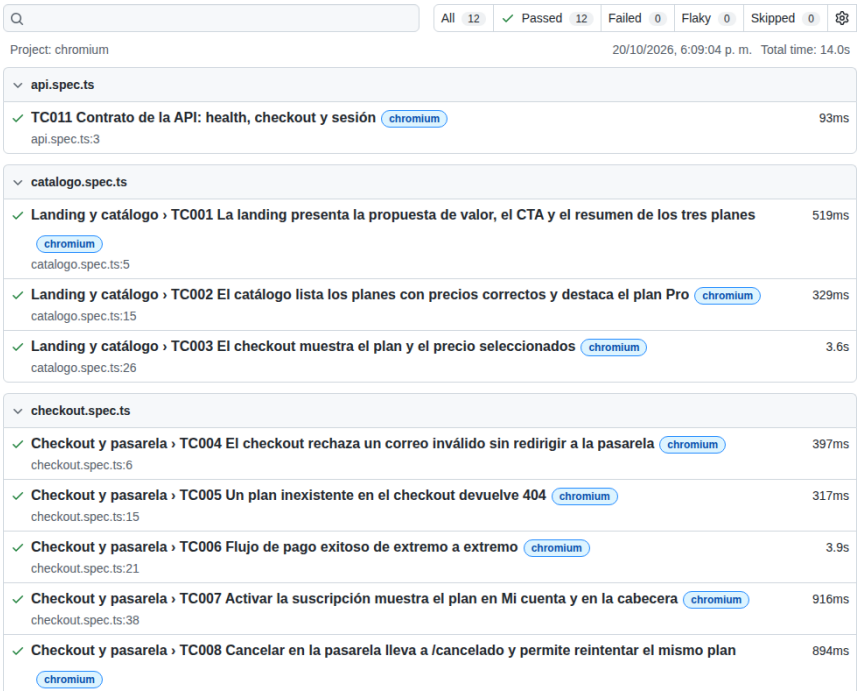



*Figura 10. Reporte HTML de Playwright de la ejecución final (por archivo y caso, con duración).*

## 5. Análisis de resultados

| Métrica | Valor |
|---|---|
| Casos del plan / implementados / automatizados | 12 / 12 / 100 % |
| Primera ejecución | 7 pasaron · 5 fallaron (45,3 s, con reintentos y trazas) |
| Ejecución final | 12 pasaron · 0 fallaron (14,0 s) |
| Hallazgos | 1 defecto de producto (TC007) · 3 expectativas incorrectas de prueba · 1 localizador ambiguo · 2 localizadores obsoletos auto-curados |
| Falsos positivos | 4 de 5 fallos iniciales fueron falsos positivos del plan (no del producto): 80 % |
| Cobertura funcional | 6/6 funcionalidades del PRD · categorías: 7 functional, 3 error handling, 1 security, 1 api |
| Evidencia por caso | Reporte HTML, traza y video en fallo, capturas de TC006/TC007, registro de curaciones |

Lectura de los resultados. El plan generado cubrió las seis funcionalidades del PRD y las tres rutas críticas del pago (éxito, cancelación, rechazo), más dos controles de seguridad que un plan manual suele omitir (sesión manipulada y activación sin pago). La tasa de falsos positivos inicial (80 %) no invalida el enfoque: todos se resolvieron en una iteración y dejaron la suite más robusta; el defecto real detectado (TC007) justifica por sí solo el ejercicio porque afectaba la primera impresión del cliente tras pagar. El tiempo de ejecución (14 s) permite usar la suite como compuerta de despliegue, que es el uso que hace el workflow de Vercel.

## 6. Soluciones técnicas y auto-curación

| Problema observado | Práctica / herramienta de auto-curación | Aplicación en el MVP |
|---|---|---|
| Localizadores que dejan de existir tras un refactor o una regeneración con IA | Localizadores con estrategias alternativas ordenadas por estabilidad (testid → rol accesible → etiqueta → texto → CSS) y registro de cada curación | tests/support/healing.ts: heal() probó la estrategia primaria, falló y resolvió por rol+nombre en TC003 y TC006; healing-log.json alimenta el mantenimiento |
| Localizadores ambiguos (strict mode) | Preferir data-testid y roles con nombre; evitar role genérico; usar .first() solo con justificación | TC004 pasó de role=alert a data-testid |
| Esperas y condiciones de carrera | Aserciones con auto-espera (expect.toBeVisible/toHaveURL), sin sleeps; retries solo en CI; trace y video en fallo | playwright.config.ts: retries=1 en CI, trace retain-on-failure |
| Dependencia de servicios externos (pasarela) | Proveedor intercambiable y sesiones firmadas; pruebas de contrato de la API; smoke en producción con datos de prueba | MockProvider en CI; Stripe test en Vercel; TC011 contrato |
| Expectativas desalineadas con el producto (moneda, textos) | Fuente única de verdad: el plan de pruebas cita literales del catálogo; formato determinista en el código | formatUsd explícito; plan y pruebas actualizados juntos |
| Regeneración de código por IA que rompe la testabilidad | Reglas del IDE (.cursor/rules) que protegen testids, roles y el proveedor mock; CI bloquea el despliegue si el plan falla | Workflow deploy-planes: e2e → deploy → smoke |
| Mantenimiento de la suite | Agentes de auto-corrección: TestSprite (reporte con causa raíz y fix sugerido), Healenium (curación por similitud del DOM para Selenium), Playwright MCP/Codegen para regenerar pasos, cuarentena de pruebas inestables | Siguiente paso: abrir un PR automático cuando healing-log registre curaciones repetidas |



```
import { test, expect } from "@playwright/test";
import { CORREO, irALaPasarela, pagarEnPasarela, pagoExitoso } from "../support/flows";
import { heal } from "../support/healing";

test.describe("Checkout y pasarela", () => {
  test("TC004 El checkout rechaza un correo inválido sin redirigir a la pasarela", async ({ page }) => {
    await page.goto("/checkout?plan=pro");
    await page.getByTestId("checkout-email").fill("correo-invalido");
    await page.getByTestId("checkout-pagar").click();
    const error = page.getByTestId("checkout-error");  // getByRole("alert") era ambiguo: Next.js añade <next-route-announcer role="alert">
    await expect(error).toHaveText("Correo electrónico inválido.");
    await expect(page).toHaveURL(/\/checkout\?plan=pro$/);
  });

  test("TC005 Un plan inexistente en el checkout devuelve 404", async ({ page }) => {
    const response = await page.goto("/checkout?plan=inexistente");
    expect(response?.status()).toBe(404);
    await expect(page.getByTestId("checkout-form")).toHaveCount(0);
  });

  test("TC006 Flujo de pago exitoso de extremo a extremo", async ({ page }, testInfo) => {
    await irALaPasarela(page, "pro");
    await expect(page.getByTestId("gateway-importe")).toContainText("USD 49");
    await expect(page.getByTestId("gateway-email")).toHaveText(CORREO);
    // El botón de pago se localiza de forma auto-curable (testid obsoleto → propósito → rol + nombre).
    const pagar = await heal(page, "botón Pagar", { testId: "btn-pay", role: { role: "button", name: /^Pagar/ } }, { test: testInfo.title });
    await page.getByTestId("gateway-card").fill("4242 4242 4242 4242");
    await pagar.click();
    await expect(page).toHaveURL(/\/exito\?session_id=/);
    await expect(page.getByTestId("exito-mensaje")).toContainText("Pago confirmado");
    await expect(page.getByTestId("exito-plan")).toHaveText("Pro");
    await expect(page.getByTestId("exito-importe")).toContainText("USD 49");
    await expect(page.getByTestId("exito-email")).toHaveText(CORREO);
    await expect(page.getByTestId("exito-referencia")).not.toBeEmpty();
    await page.screenshot({ path: "test-results/evidencia/TC006_exito.png", fullPage: true });
  });

  test("TC007 Activar la suscripción muestra el plan en Mi cuenta y en la cabecera", async ({ page }) => {
    await pagoExitoso(page, "pro");
    await page.getByTestId("activar-suscripcion").click();
    await expect(page).toHaveURL(/\/cuentas$/);
    await expect(page.getByTestId("cuenta-plan")).toHaveText("Pro");
    await expect(page.getByTestId("cuenta-email")).toHaveText(CORREO);
    await expect(page.getByTestId("nav-plan-activo")).toHaveText("Plan Pro");
    await page.screenshot({ path: "test-results/evidencia/TC007_cuenta.png", fullPage: true });
  });
});
```

```
# Despliegue continuo de AsesorIA Planes (Unidad 4): E2E con pasarela simulada → Vercel (producción) → smoke en la URL pública.
# Secretos: VERCEL_TOKEN, VERCEL_ORG_ID, VERCEL_PROJECT_ID. Variables de Stripe se configuran en el proyecto de Vercel.
name: deploy-planes

on:
  push:
    branches: [main]
    paths: ["asesoria-planes/**", ".github/workflows/deploy-planes.yml"]
  pull_request:
    paths: ["asesoria-planes/**"]
  workflow_dispatch:

defaults:
  run:
    working-directory: asesoria-planes

jobs:
  e2e:
    name: Build + E2E (Playwright, pasarela mock)
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm, cache-dependency-path: asesoria-planes/package-lock.json }
      - run: npm ci
      - run: npx playwright install --with-deps chromium
      - run: npm run lint && npx tsc --noEmit
      - name: Build (proveedor mock)
        run: npm run build
        env: { PAYMENTS_PROVIDER: mock }
      - name: Pruebas E2E (12 casos del plan TestSprite)
        run: npx playwright test
        env: { PAYMENTS_PROVIDER: mock, CI: "true" }
      - name: Resumen
        if: always()
        run: |
          echo "### E2E AsesorIA Planes" >> "$GITHUB_STEP_SUMMARY"
          node -e 'const r=require("./test-results/results.json");const s=r.stats;console.log(`| Pasaron | Fallaron | Omitidas | Duración |\n|---|---|---|
          if [ -f test-results/healing-log.json ]; then { echo; echo "Curaciones de localizadores:"; echo '```'; cat test-results/healing-log.json; ec
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: playwright-report, path: "asesoria-planes/playwright-report\nasesoria-planes/test-results" }

  deploy:
```

*Figura 11. Implementación del plan y del despliegue: tests/e2e/checkout.spec.ts (TC006–TC008 con heal()) y .github/workflows/deploy-planes.yml (job e2e como compuerta del deploy).*

**[PENDIENTE] Captura de Stripe Checkout (modo test)**

Página alojada de Stripe con el plan Pro (USD 49/mes) y el pago de prueba 4242… aprobado; Payments del dashboard en modo test. Guía §2.

*Figura 12. Stripe Checkout en modo test sobre el despliegue (captura pendiente; ver Anexo).*

**[PENDIENTE] Captura de GitHub Actions**

Workflow deploy-planes en verde: e2e → deploy → smoke-prod, con el resumen de E2E y la URL desplegada.

*Figura 13. Pipeline deploy-planes: E2E → Vercel → smoke (captura pendiente; ver Anexo).*

## 7. Conclusiones

El ejercicio confirma que la automatización E2E con agentes de IA funciona mejor cuando el producto se construye para ser probado: identificadores estables, nombres accesibles, un proveedor de pagos simulado y una API de contrato convirtieron un flujo de pago, normalmente frágil, en una suite determinista de 14 segundos que corre antes de cada despliegue. El agente

generó un plan completo (funcional, errores, seguridad, API) a partir del PRD, pero la primera ejecución mostró que el 80 % de los fallos eran falsos positivos: expectativas de formato y localizadores ambiguos. Ese es el riesgo principal de las pruebas generadas por IA y la razón de revisar cada caso antes de darlo por válido; a cambio, el mismo plan encontró un defecto real de la aplicación (la cabecera no reflejaba la suscripción) que una revisión manual habría pasado por alto.

Las técnicas de auto-curación reducen el mantenimiento, pero deben ser auditables: el helper registra cada curación y la CI la publica, de modo que una curación no oculte un cambio funcional. En un contexto bancario esto importa tanto como la cobertura: una prueba que «se cura sola» sin dejar rastro es un control que ya no verifica lo que decía verificar. La integración con TestSprite aporta el siguiente nivel (generación, ejecución en la nube y propuesta de corrección), y la arquitectura del MVP está lista para ejecutarla contra el despliegue en Vercel con Stripe en modo test.

## Anexo. Pasos pendientes con cuentas propias

- Hecho: proyecto generado y refinado en Cursor (Figuras 3, 4, 6, 11); despliegue en Vercel con proveedor mock en https://asesoria-planes.vercel.app (Figura 5).
- Stripe: crear claves de prueba y el webhook; configurar STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET y PAYMENTS_PROVIDER=stripe en Vercel y redesplegar (guía §2–3).
- GitHub: añadir VERCEL_TOKEN / VERCEL_ORG_ID / VERCEL_PROJECT_ID como secretos para que deploy-planes complete los jobs deploy y smoke-prod (guía §3).
- TestSprite: configurar el MCP con la API key y ejecutar el plan desde Cursor contra localhost:3001 (mock) y contra la URL de Vercel (guía §4); guardar testsprite-mcp-test-report.md y las capturas.
- Sustituir las figuras 7, 12 y 13 por las capturas de TestSprite, Stripe Checkout y GitHub Actions, y añadir la comparación TestSprite vs. Playwright a la tabla de métricas.